Chapter 8 exercises

DS08-E01 • Why tombstones are needed

With fixed m=7 and automatic rebuilding disabled, h(k)=k mod 7, use linear probing to insert 10,17,24 and delete 17. Draw slot states and trace searching for 24 and inserting 31. Why must insertion continue duplicate checking after the first Deleted slot?

DS08-E02 • Probe-sequence coverage

For double hashing with capacity 8 and start 1, list full cycles with steps 2 and 3. Derive the necessary and sufficient coprimality condition for full coverage. What can cause insertion failure while empty slots remain?

DS08-E03 • Conditions of expectation formulas

Use the ideal uniform-hashing model with a table built only by insertion, without deletion. A successful query selects each of the n stored keys with equal probability. Compute unsuccessful-search and average successful-search upper bounds for $\alpha=0.5$ and $\alpha=0.9$. Why are these not exact averages for linear probing with tombstones?

DS08-E04 • Resize oscillation

A table doubles when $\alpha>1/2$ and halves when $\alpha<1/2$. Construct updates that rebuild repeatedly near the same element count. Propose distinct thresholds and explain how intervening updates pay for rebuilding.

DS08-E05 • Hash families, collision pairs, and quantifiers

Keys are vectors in F_p^2. Choose a,b independently uniformly and use h(x)=a x1+b x2 mod p. Prove pair collision probability and derive whole-table expected collision pairs for n keys fixed before choosing h. Compare a fixed unsuccessful query. Does merely translating g(x)=x1 give the same guarantee? For p=11 and keys (0,0),(1,0),(0,1),(1,1), compute the exact expectation and explain function a=b=0.

DS08-E06 • Wraparound, duplicate checking, and cleanup

> Use m=8, h(k,i)=(k+i) mod 8, fixed capacity with automatic rebuilding disabled. N=Never-used, D=Deleted, O=Occupied. Insert/update preserves unique keys. Insert (6,A),(14,B),(22,C), then erase 14.
>
> 1. Give all eight slot states and the complete find-22 trace. Where would search incorrectly stop if deletion produced N?
> 2. Update (22,C*), then insert (30,Z). For each, give probes, first Deleted candidate, stopping reason, written slot, n, and d. Which two slots would duplicate 22 if update wrote at the first D?
> 3. Separately insert 6,14,22,1,2,3,4,5 into an empty table, then erase every key except 22. How many probes find absent 38, and which insertion slot is returned? What if all slots are Occupied with other keys?
> 4. Rehash the third case at the same capacity. Give 22's new slot and the new find-38 path. Why not merely turn D into N? Can n=1,d=7 be treated as a retained state of the chapter's automatic policy?
> 5. Prove that insertion, deletion, and D reuse preserve the invariant that no live key has N before it in its probe sequence. What extra precondition lets insert_new write at the first D?

DS08-E07 • Collision, overflow and load

There are B=3 buckets with S=2 fixed slots each and h(k)=k mod 3. Insert 1,4,7 in order. When does a collision occur, and when is overflow handling needed? What is fixed-slot load after inserting 4? If only 1 and 4 are stored in three chaining buckets, what is average chain length?

DS08-E08 • Quadratic probing and the prime condition

For m=7 and $h(k,i)=(h_0+i^2) \bmod 7$ with $h_0=0$, list probes i=0..6 and the distinct slots. If 0,1,2,4 are full and the rest empty, can another same-home key be inserted? Prove the first (p+1)/2 positions are distinct for odd prime p and give a sufficient occupied-count condition for a free position.

DS08-E09 • Full cycles and probe assumptions

With m=12 and home=1, list cycles for steps 4 and 5 and their lengths. Prove the general length m/gcd(m,s). Does full coverage justify the expected-probe formula for ideal random permutations?

DS08-E10 • Hash design and the term rehash

Square 8125, write the eight-digit result, select its middle four digits, and reduce the number modulo 101. Split 123203241112020 into three-digit groups, sum and reduce modulo 100. Why do these designs not automatically establish universal hashing? Distinguish a subsequent probe from rebuilding a whole table.

References and attribution

Lin Cing, handwritten data structures notes; MIT 6.006 hashing notes; ISO C++ unordered-container requirements.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.
https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study